

과정2-4번

1.ROS2 환경설정

ROS2 명령어 사용전 현재터미널에 ROS2 Humble 환경 적용

```
source /opt/ros/humble/setup/bash
```

매번 터미널을 열때마다 자동으로 적용하려면 다음 명령실행

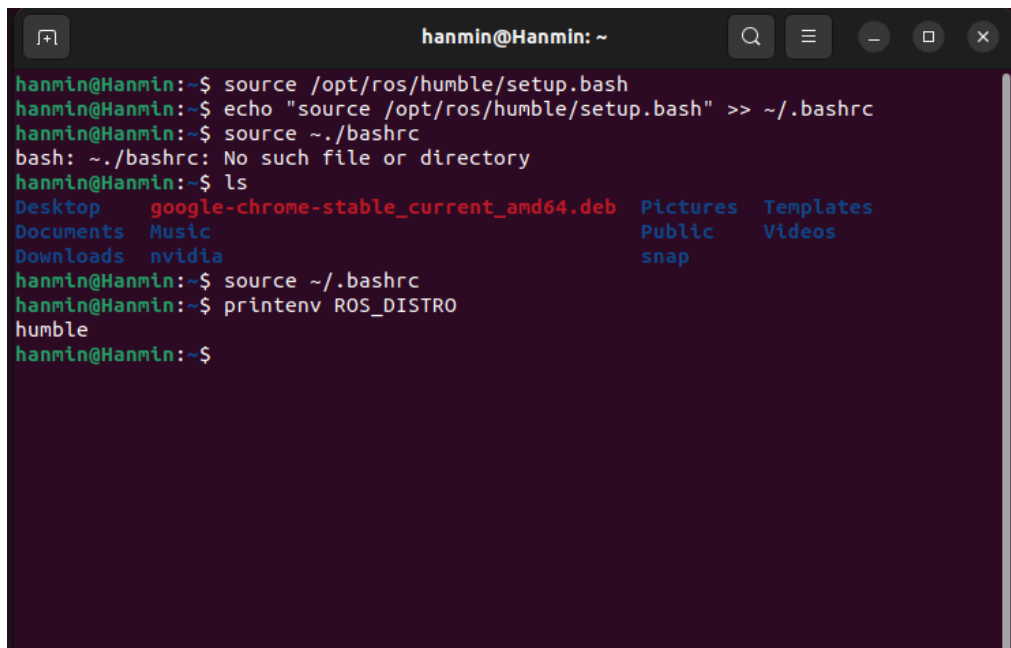
```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

설정 여부 확인방법

```
printenv ROS_DISTRO
```

정상적으로 ROS2를 설치한경우

```
humble
#이렇게 뜨게된다.
#만약 ros2 comman not found 또는 ROS2 패키지를 찾을수 없다는 오류가 뜬다면
/opt/ros/humble/setup.bash 를 source 하지 않았을 가능성이 크다.
```

A terminal window titled 'hanmin@Hanmin: ~' showing the following commands and output:

```
hanmin@Hanmin:~$ source /opt/ros/humble/setup.bash
hanmin@Hanmin:~$ echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
hanmin@Hanmin:~$ source ~/.bashrc
bash: ~/.bashrc: No such file or directory
hanmin@Hanmin:~$ ls
Desktop      google-chrome-stable_current_amd64.deb  Pictures  Templates
Documents    Music                                   Public    Videos
Downloads    nvidia                                  snap
hanmin@Hanmin:~$ source ~/.bashrc
hanmin@Hanmin:~$ printenv ROS_DISTRO
humble
hanmin@Hanmin:~$
```

2.ROS2_워크스페이스 생성

워크스페이스가 뭐예요?(Work Space)

ROS2 워크스페이스는 하나 이상의ROS2 패키지를 개발하고 빌드하기위한 작업공간이다.

일반적으로 워크스페이스 루트 아래에 src 디렉토리를 만들고, 개발할 패키지는 모두 src내부에 배치한다.

즉 ,워크스페이스는 작업공간이라는 의미이며 너가짜고 동작하는 모든 코드들은 다 워크스페이스 안에서 이뤄져야지 나중에 ROS2 에서 인식해서 동작할 수 있다. 라는 의미이다.

디렉토리 생성

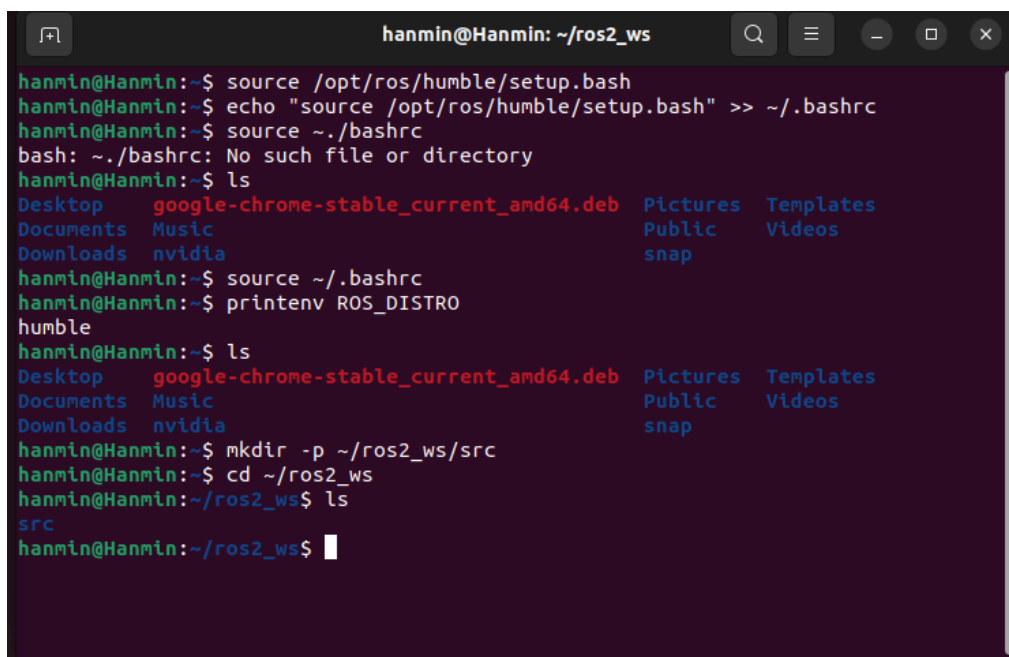
```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws
```

생성결과는 다음과 같다

```
디렉토리 파일 구성 (tree 형태)
ros2_ws/
|
-----src/
```

mkdir -p 에서 -p옵션은 상위 디렉토리가 없을 경우 함께 생성하고, 이미디렉토리가 존재하더라도 오류를 발생시키지 않는다.

즉 ros2_ws 를 생성하고 그안에 src폴더를 생성해야 하는데 ros2_ws 관련된 디렉토리가 존재하지 않는다면? 먼저 생성하고그다음 src폴더를 생성하는것



```
hanmin@Hanmin: ~/ros2_ws
hanmin@Hanmin:~$ source /opt/ros/humble/setup.bash
hanmin@Hanmin:~$ echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
hanmin@Hanmin:~$ source ~/.bashrc
bash: ~/.bashrc: No such file or directory
hanmin@Hanmin:~$ ls
Desktop      google-chrome-stable_current_amd64.deb  Pictures  Templates
Documents    Music                                   Public    Videos
Downloads    nvidia                                  snap
hanmin@Hanmin:~$ source ~/.bashrc
hanmin@Hanmin:~$ printenv ROS_DISTRO
humble
hanmin@Hanmin:~$ ls
Desktop      google-chrome-stable_current_amd64.deb  Pictures  Templates
Documents    Music                                   Public    Videos
Downloads    nvidia                                  snap
hanmin@Hanmin:~$ mkdir -p ~/ros2_ws/src
hanmin@Hanmin:~$ cd ~/ros2_ws
hanmin@Hanmin:~/ros2_ws$ ls
src
hanmin@Hanmin:~/ros2_ws$
```

3.colcon 조사

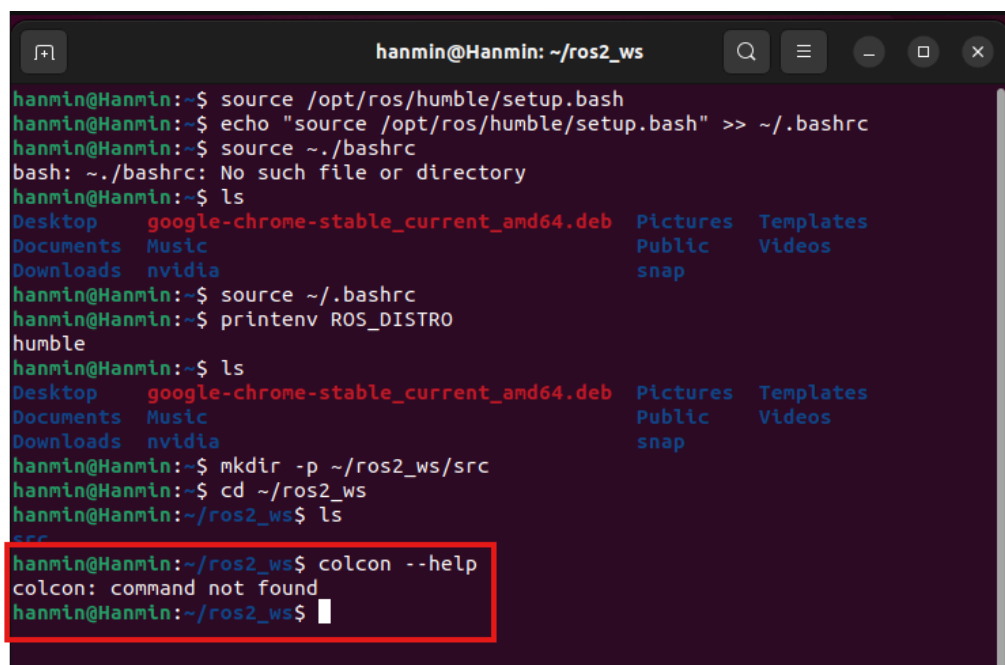
colcon의 개념

colcon은 ROS2 워크스페이스 안의 여러 패키지를 검색, 패키지 간 의존관계를 고려, 적절한 순서를 빌드하는 명령행 도구이다.

ROS2 패키지 자체의 빌드 방식은 ament_cmake, ament_python 등으로 나뉘지만, 워크스페이스 전체 빌드는 일반적으로 colcon 이 담당

colcon 설치확인

```
colcon --help
```



```
hanmin@Hanmin: ~/ros2_ws
hanmin@Hanmin:~$ source /opt/ros/humble/setup.bash
hanmin@Hanmin:~$ echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
hanmin@Hanmin:~$ source ~/.bashrc
bash: ~/.bashrc: No such file or directory
hanmin@Hanmin:~$ ls
Desktop      google-chrome-stable_current_amd64.deb  Pictures  Templates
Documents    Music                                   Public    Videos
Downloads    nvidia                                  snap
hanmin@Hanmin:~$ source ~/.bashrc
hanmin@Hanmin:~$ printenv ROS_DISTRO
humble
hanmin@Hanmin:~$ ls
Desktop      google-chrome-stable_current_amd64.deb  Pictures  Templates
Documents    Music                                   Public    Videos
Downloads    nvidia                                  snap
hanmin@Hanmin:~$ mkdir -p ~/ros2_ws/src
hanmin@Hanmin:~$ cd ~/ros2_ws
hanmin@Hanmin:~/ros2_ws$ ls
src
hanmin@Hanmin:~/ros2_ws$ colcon --help
colcon: command not found
hanmin@Hanmin:~/ros2_ws$
```

만약패키지가 다운받아지지 않을경우가 존재한다 그럴때는

```
sudo apt update
sudo apt install python3-colcon-common-extensions
```

```
hanmin@Hanmin: ~/ros2_ws
Building dependency tree... Done
Reading state information... Done
21 packages can be upgraded. Run 'apt list --upgradable' to see them.
hanmin@Hanmin:~/ros2_ws$ sudo apt install python3-colcon-common-extensions
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  libfwupd2 libfwupdplugin5 libgcab-1.0-0 libsbios-c2
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  libjs-jquery-hotkeys libjs-jquery-isonscreen libjs-jquery-metadata
  libjs-jquery-tablesorter libjs-jquery-throttle-debounce
  python3-colcon-argcomplete python3-colcon-bash python3-colcon-cd
  python3-colcon-cmake python3-colcon-core python3-colcon-defaults
  python3-colcon-devtools python3-colcon-installed-package-information
  python3-colcon-library-path python3-colcon-metadata
  python3-colcon-notification python3-colcon-output
  python3-colcon-override-check python3-colcon-package-information
  python3-colcon-package-selection python3-colcon-parallel-executor
  python3-colcon-pkg-config python3-colcon-powershell
  python3-colcon-python-setup-py python3-colcon-recursive-crawl
  python3-colcon-ros python3-colcon-test-result python3-colcon-zsh
  python3-cov-core python3-coverage python3-distlib python3-nose2
```

설치확인

```
colcon --help
```

```
hanmin@Hanmin: ~/ros2_ws
hanmin@Hanmin:~/ros2_ws$ colcon --help
usage: colcon [-h] [--log-base LOG_BASE] [--log-level LOG_LEVEL]
             {build,extension-points,extensions,graph,info,list,metadata,test,
             test-result,version-check}
...
options:
  -h, --help                show this help message and exit
  --log-base LOG_BASE       The base path for all log directories (default: ./log,
                             to disable: /dev/null)
  --log-level LOG_LEVEL     Set log level for the console output, either by
                             numeric or string value (default: warning)

colcon verbs:
  build                     Build a set of packages
  extension-points          List extension points
  extensions                List extensions
  graph                     Generate a visual representation of the dependency graph
  info                     Package information
  list                      List packages, optionally in topological ordering
  metadata                  Manage metadata of packages
  test                     Test a set of packages
  test-result               Show the test results generated when testing a set of
```

빈 워크스페이스 빌드

워크스페이스 루트에서 다음 명령을 실행

```
cd ~/ros2_ws
colcon build
```

```
hanmin@Hanmin: ~/ros2_ws
COLCON_EXTENSION_BLOCKLIST
Block extensions which should not be used
COLCON_HOME
Set the configuration directory (default: ~/.colcon)
COLCON_LOG_LEVEL
Set the log level (debug|10, info|20, warn|30,
error|40, critical|50, or any other positive numeric
value)
COLCON_WARNINGS
Set the warnings filter similar to PYTHONWARNINGS
except that the module entry is implicitly set to
'colcon.*'
CTEST_COMMAND
The full path to the CTest executable
POWERSHELL_COMMAND
The full path to the PowerShell executable
For more help and usage tips, see https://colcon.readthedocs.io
hanmin@Hanmin:~/ros2_ws$
hanmin@Hanmin:~/ros2_ws$
hanmin@Hanmin:~/ros2_ws$
hanmin@Hanmin:~/ros2_ws$ ;s
bash: syntax error near unexpected token `;'
hanmin@Hanmin:~/ros2_ws$ ls
src
hanmin@Hanmin:~/ros2_ws$ colcon build
Summary: 0 packages finished [0.58s]
hanmin@Hanmin:~/ros2_ws$
```



주의할점 :

colcon build가 패키지를 생성하는 명령은 아니라는것
이 명령은 src에서 패키지를 검색하여 빌드한다.

src가 비어 있다면 빌드할 패키지가 없으며, 환경에 따라 build, install, log 디렉토리 생성

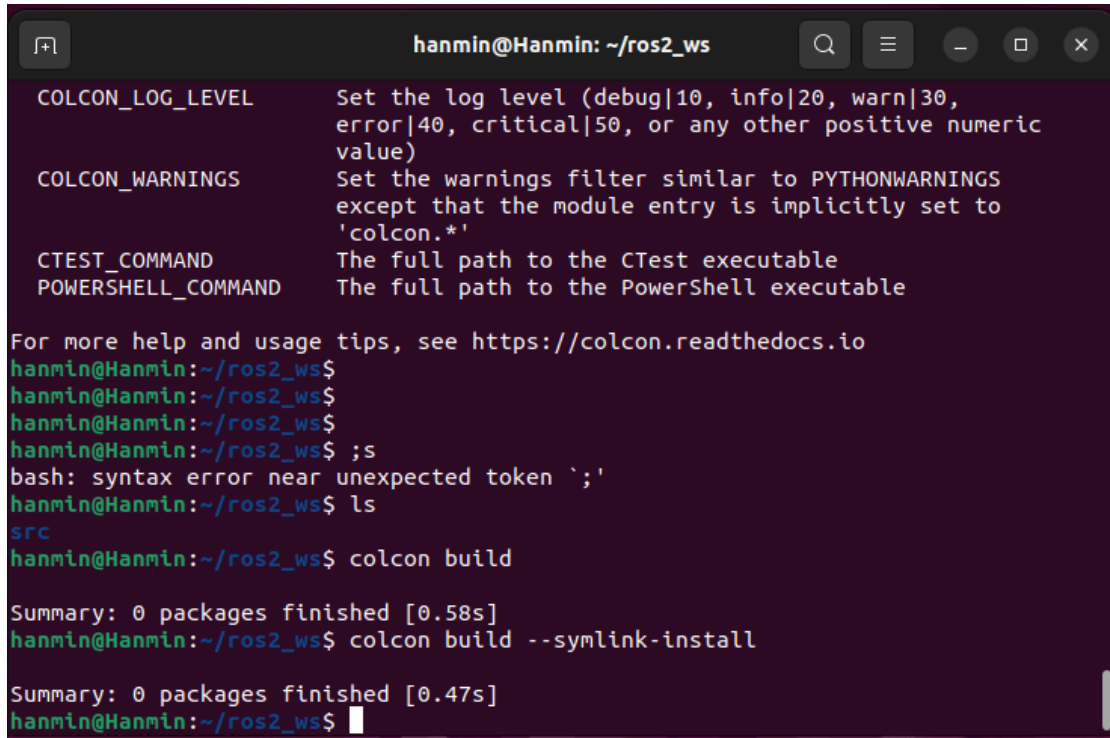
Ros2패키지는 ros2 pkg create 명령으로 생성됨

주요 colcon 명령 및 옵션

명령 또는 옵션	설명
colcon build	워크스페이스의 패키지를 빌드
colcon test	빌드된 패키지의 테스트 수행
colcon test-result	테스트 결과를 확인
—packages-select [패키지명]	지정한 패키지만 빌드
—packages-up-to [패키지명]	지정 패키지와 그 의존 패키지를 함께 빌드
—symlink-install	python 파일등을 복사하지 않고 심볼릭 링크를 설치하여 수정내용을 빠르게 반영
—event-handlers console_direct+	빌드 로그를 터미널에 바로 자세히 출력

개발중에는 다음 명령이 편리하다(심볼릭 링크 == 윈도우 바로가기 아이콘)

```
colcon build --symlink-install
```



```
hanmin@Hanmin: ~/ros2_ws
COLCON_LOG_LEVEL      Set the log level (debug|10, info|20, warn|30,
                      error|40, critical|50, or any other positive numeric
                      value)
COLCON_WARNINGS       Set the warnings filter similar to PYTHONWARNINGS
                      except that the module entry is implicitly set to
                      'colcon.*'
CTEST_COMMAND         The full path to the CTest executable
POWERSHELL_COMMAND    The full path to the PowerShell executable

For more help and usage tips, see https://colcon.readthedocs.io
hanmin@Hanmin:~/ros2_ws$
hanmin@Hanmin:~/ros2_ws$
hanmin@Hanmin:~/ros2_ws$
hanmin@Hanmin:~/ros2_ws$ ;s
bash: syntax error near unexpected token `;'
hanmin@Hanmin:~/ros2_ws$ ls
src
hanmin@Hanmin:~/ros2_ws$ colcon build

Summary: 0 packages finished [0.58s]
hanmin@Hanmin:~/ros2_ws$ colcon build --symlink-install

Summary: 0 packages finished [0.47s]
hanmin@Hanmin:~/ros2_ws$
```

ROS2 패키지 생성

기본형식은 다음과 같다

```
ros2 pkg create [옵션] 패키지명
```

주요옵션

옵션	설명
—build-type	사용할 빌드 시스템을 지정
—dependencies	패키지가 의존하는 ROS2패키지를 저장
—node-name	패키지 생성 시 기본 노드 파일도 함께 만들
—license	패키지 라이선스를 지정
—description	패키지 설명을 입력
—maintainer-name	관리자 이름을 지정
—maintainer-email	관리자 이메일을 지정

명령어 도움말은 다음과 같이 확인한다.

```
ros2 pkg create --help
```

```
hanmin@Hanmin: ~/ros2_ws
Summary: 0 packages finished [0.47s]
hanmin@Hanmin:~/ros2_ws$ ros2 pkg create --help
usage: ros2 pkg create [-h] [--package-format {2,3}]
                        [--description DESCRIPTION] [--license LICENSE]
                        [--destination-directory DESTINATION_DIRECTORY]
                        [--build-type {cmake,ament_cmake,ament_cargo,ament_python
                        }]
                        [--dependencies DEPENDENCIES [DEPENDENCIES ...]]
                        [--maintainer-email MAINTAINER_EMAIL]
                        [--maintainer-name MAINTAINER_NAME]
                        [--node-name NODE_NAME] [--library-name LIBRARY_NAME]
                        package_name

Create a new ROS 2 package

positional arguments:
  package_name          The package name

options:
  -h, --help            show this help message and exit
  --package-format {2,3}, --package_format {2,3}
                        The package.xml format.
  --description DESCRIPTION
                        The description given in the package.xml
  --license LICENSE      The license attached to this package; this can be an
                        arbitrary string, but a LICENSE file will only be
                        generated if it is one of the supported licenses (pass
                        '?' to get a list)
  --destination-directory DESTINATION_DIRECTORY
                        Directory where to create the package directory
  --build-type {cmake,ament_cmake,ament_cargo,ament_python}
                        The build type to process the package with
  --dependencies DEPENDENCIES [DEPENDENCIES ...]
                        list of dependencies
  --maintainer-email MAINTAINER_EMAIL
                        email address of the maintainer of this package
  --maintainer-name MAINTAINER_NAME
                        name of the maintainer of this package
  --node-name NODE_NAME
                        name of the empty executable
  --library-name LIBRARY_NAME
```

my_robot_controller 패키지 생성

ROS2 패키지는 워크스페이스 루트가 아니라 src 디렉토리에서 생성

```
#여러줄 버전
cd ~/ros2_ws/src
ros2 pkg create \
  --build-type ament_python \
  --dependencies rclpy \
  --license Apache-2.0 \
  my_robot_controller

#한줄버전
```

```
ros2 pkg create --build-type ament_python --dependencies rclpy --license Apache-2.0 my_robot_controller
```

이 명령은 다음 내용을 의미

- 패키지 이름:my_robot_controller
- 빌드시스템 : ament_python
- 의존 패키지:rclpy
- 라이선스:Apache-2.0

—dependencies rclpy를 지정하면 package.xml에 rclpy의존성이 자동으로 추가된다.

ament_python 빌드 시스템

개념

ament는 ROS2 패키지의 빌드와 설치를 지원하는 빌드 시스템 및 관련 도구의 집합

ROS2에서 대표적을 사용하는 빌드 유형은 다음과 같다

빌드 유형	주 사용 언어 및 용도
ament_cmake	C++ 중심 ROS2 패키지
ament_python	순수 python 중심 ROS2 패키지
ament_cmake_python	Cmake 패키지 내부에서 Python 코드도 함께 설치할 때 사용

ament_python 은 python 의 setuptools 구조를 기반으로 ROS2 Python 패키지를 설치한다. 따라서 패키지 루트에 setup.py, setup.cfg, package.xml등이 생성된다.

ament_python을 사용하는 이유

- Python으로 ROS2 노드를 쉽게 개발할 수 있다.
- Python 표준 패키징 방식인 setuptools를 활용
- setup.py의 console_scripts를 통해 ros2 run 으로 실행 가능한 명령을 등록할 수 있다.
- colcon build와 연동되어 워크스페이스 단위로 패키지를 빌드할 수 있다
- —symlink-install 을 사용하면 Python 소스 수정결과를 빠르게 확인 가능하다.

rclpy 조사

rclpy란?

rclpy는 ROS2 에서 Python 프로그램이 ROS2 기능을 사용할 수 있게 해주는 Python 클라이언트 라이브러리이다.

C++에서 사용하는 rclcpp 와 대응되는 Python 용 라이브러리이다.

rclpy의 주요 용도

rclpy를 사용하면 Python 코드에서 다음기능을 구현할 수 있다.

- ROS2 노드 생성
- 토픽 Publisher 생성
- 토픽 Subscriber 생성
- Service 서버 및 Client 생성
- Action 서버 및 Client 생성
- Parameter 선언 및 사용
- Timer와 Callback 실행
- Logger를 통한 로그 출력
- ROS 2 Context 초기화 및 종료
- Executor 를 통한 Callback 처리

rclpy 의 설명

- **노드 생성**: ROS 2에서 동작하는 하나의 프로그램을 만든다.
- **Publisher**: 다른 프로그램에 데이터를 보낸다.
- **Subscriber**: 다른 프로그램이 보낸 데이터를 받는다.
- **Service 서버·Client**: 요청을 보내고 한 번의 결과를 돌려받는다.
- **Action 서버·Client**: 시간이 오래 걸리는 작업을 요청하고 진행 상황과 결과를 받는다.
- **Parameter**: 속도나 기준값 같은 설정값을 저장하고 변경한다.
- **Timer**: 일정 시간마다 작업을 반복한다.
- **Callback**: 데이터나 요청이 들어왔을 때 실행할 함수를 정한다.
- **Logger**: 실행 상태나 오류를 터미널에 출력한다.
- **Context 초기화·종료**: ROS 2 통신을 시작하고 안전하게 끝낸다.
- **Executor**: 여러 Callback이 실행될 순서와 방식을 관리한다.

한 문장으로 정리하면:

`rclpy` 는 Python 프로그램이 ROS 2 안에서 데이터를 주고받고, 명령을 처리하며, 반복 작업을 수행할 수 있게 해주는 라이브러리이다.

기본 사용 흐름

```
import rclpy
from rclpy.node import Node
```

```

class RobotController(Node):

    def __init__(self):
        super().__init__('robot_controller')
        self.get_logger().info('제어 노드가 시작되었습니다.')

def main(args=None):
    rclpy.init(args=args)

    node = RobotController()
    rclpy.spin(node)

    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

주요 함수와 클래스의 역할

항목	역할
rclpy.init()	ROS 2 Python 통신 환경을 초기화한다.
Node	ROS 2 노드를 구현하기 위한 기본 클래스이다.
rclpy.spin(node)	노드를 계속 실행하며 콜백을 처리
destroy_node()	노드가 사용한 자원을 해제한다.
rclpy.shutdown()	ROS2 Python 환경을 종료한다.
get_logger()	ROS 2 로그 메시지를 출력한다.

생성된 패키지 구조

my_robot_controller 패키지를 생성하면 기본적으로 다음과 유사한 구조가 생성된다.

```

my_robot_controller/
├── my_robot_controller/
│   └── __init__.py
├── package.xml
├── resource/
│   └── my_robot_controller
├── setup.cfg
├── setup.py
├── test/
│   ├── test_copyright.py
│   ├── test_flake8.py
│   └── test_pep257.py

```

본 제출용 예시에는 실행 확인을 위한 controller_node.py 도 추가하였다.

```
my_robot_controller/  
├── my_robot_controller/  
│   ├── __init__.py  
│   └── controller_node.py  
├── package.xml  
├── resource/  
│   └── my_robot_controller  
├── setup.cfg  
├── setup.py  
└── test/  
    ├── test_copyright.py  
    ├── test_flake8.py  
    └── test_pep257.py
```

package.xml 파일

package.xml의 역할

package.xml은 ROS2 패키지의 메타데이터와 의존성을 기록하는 패키지 매니페스트 파일이다.

모든 ament패키지는 패키지 루트에 하나의 package.xml을 가져야 한다.

주요 기록 내용은 다음과 같다.

- 패키지 이름
- 버전
- 설명
- 관리자 이름과 이메일
- 라이선스
- 빌드 및 실행 의존성
- 테스트 의존성
- 빌드 시스템 유형

package.xml 예시

```
<?xml version="1.0"?>  
<package format="3">  
  <name>my_robot_controller</name>  
  <version>0.0.0</version>  
  <description>ROS 2 Python 로봇 제어 패키지</description>
```

```

<maintainer email="kdg5794@gmail.com">Kim Hanmin</maintainer>
<license>Apache-2.0</license>

<depend>rcipy</depend>

<test_depend>ament_copyright</test_depend>
<test_depend>ament_flake8</test_depend>
<test_depend>ament_pep257</test_depend>
<test_depend>python3-pytest</test_depend>

<export>
  <build_type>ament_python</build_type>
</export>
</package>

```

주요 태그

태그	역할
<name>	패키지이름
<version>	패키지버전
<description>	패키지 설명
<maintainer>	유지관리자 이름과 이메일
<license>	적용 라이선스
<depend>	빌드와 실행에 모두 필요한 의존성
<build_depend>	빌드시 필요한 의존성
<exec_depend>	실행시 필요한 의존성
<test_depend>	테스트 수행시 필요한 의존성
<export>	빌드 시스템 등 추가 정보를 내보내는 영역
<build_type>	패키지의 빌드유형

```

<depend>rcipy</depend>

```

Python 패키지임을 나타내기 위해 다음 항목이 필요하다.

```

<export>
  <build_type>ament_python</build_type>
</export>

```

setup.py파일

setup.py의 역할

setup.py는 Python 패키지의 설치 방법과 실행가능한 노드 등록 방법을 정의한다.

setup.py 는 Python 패키지의 설치 방법과 실행 가능한 노드 등록 방법을 정의한다.

ROS 2에서 ament_python 패키지를 빌드할 때 colcon 은 setup.py 정보를 사용하여 다음 작업을 수행한다.

- Python 모듈 설치
- package.xml 및 resource 파일 설치
- 패키지 이름과 버전 등록
- 의존 Python 패키지 지정
- ros2 run 에서 사용할 실행 명령 등록

setup.py 예시

```
from setuptools import find_packages, setup

package_name = 'my_robot_controller'

setup(
    name=package_name,
    version='0.0.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        (
            'share/ament_index/resource_index/packages',
            ['resource/' + package_name]
        ),
        (
            'share/' + package_name,
            ['package.xml']
        ),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='Kim Hanmin',
    maintainer_email='student@example.com',
    description='ROS 2 Python 로봇 제어 패키지',
    license='Apache-2.0',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
            'controller_node = my_robot_controller.controller_node:main',
        ],
    },
)
```

주요항목

항목	역할
name	설치되는 파이썬 패키지이름
version	패키지버전
packages	설치할 파이썬 모듈검색
data_files	package.xml, resource 하일등의 설치 위치 지정
install_requires	파이썬 패키지 설치 의존성
maintainer	관리자 이름
maintainer_email	관리자 이메일
description	패키지 설명
license	라이선스
tests_require	테스트에 필요한 파이썬 패키지
entry_points	실행가능한 콘솔 명령 등록

다음 설정이 가장 중요

```
entry_points={
    'console_scripts': [
        'controller_node = my_robot_controller.controller_node:main',
    ],
},
```

이 설정은 다음 관계를 의미

```
ros2 run에서 사용할 실행 이름
↓
controller_node
↓
Python 모듈
my_robot_controller.controller_node
↓
실행 함수
main
```

빌드 후에는 다음 명령으로 노드를 실행할 수 있다.

```
ros2 run my_robot_controller controller_node
```

setup.cfg파일

`setup.cfg` 는 ROS 2가 Python 실행 스크립트를 패키지별 경로에 설치하도록 지정한다.

```
[develop]
script_dir=$base/lib/my_robot_controller

[install]
install_scripts=$base/lib/my_robot_controller
```

이 파일이 올바르게 설정되어야 `ros2 run 패키지명 실행파일명` 형식으로 Python 노드를 찾을 수 있다.

resource 디렉토리

```
resource/
└─ my_robot_controller
```

resource 파일은 보통 내용이 없는 빈 파일이지만, ROS 2의 ament index가 설치된 패키지를 검색할 수 있도록 패키지 이름을 등록하는 역할을 한다.

`setup.py`의 다음 부분이 resource 파일의 설치 위치를 지정한다.

```
(
    'share/ament_index/resource_index/packages',
    ['resource/' + package_name]
),
```

패키지 빌드

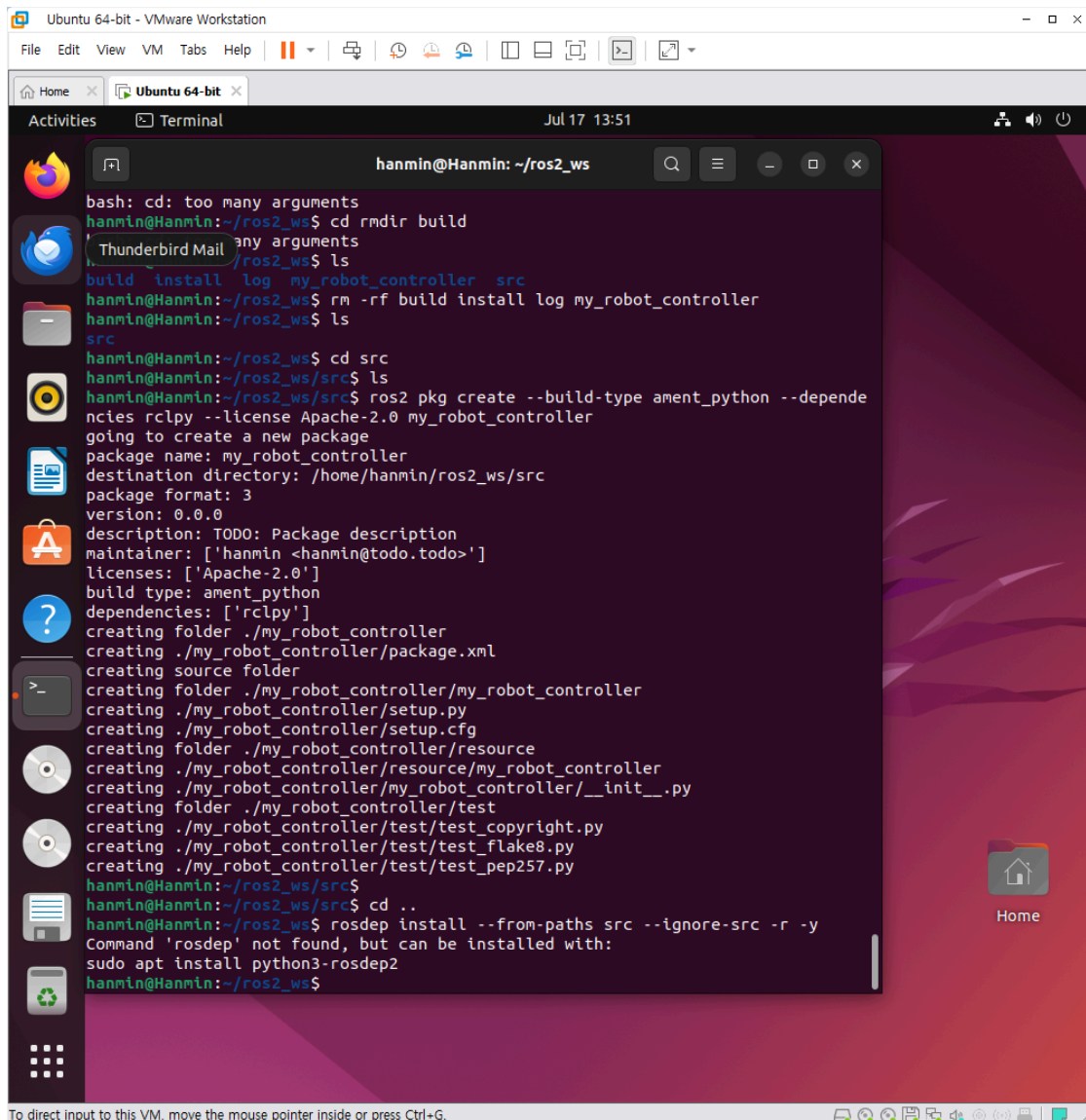
패키지를 만든 후 워크스페이스 루트로 이동

```
cd ~/ros2_ws
```

```
sudo apt install python3-rosdep2 #ROS2 의존성 설치
```

```
rosdep update
```

```
#한다음
```




```
hanmin@Hanmin: ~/ros2_ws
hanmin@Hanmin:~/ros2_ws$ rosdep install --from-paths src --ignore-src -r -y
Command 'rosdep' not found, but can be installed with:
sudo apt install python3-rosdep2
hanmin@Hanmin:~/ros2_ws$ sudo apt install python3-rosdep2
[sudo] password for hanmin:
Sorry, try again.
[sudo] password for hanmin:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  libfwupd2 libfwupdplugin5 libgcab-1.0-0 libsmbios-c2
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  python3-catkin-pkg python3-roscdistro python3-rospkg
The following NEW packages will be installed:
  python3-catkin-pkg python3-rosdep2 python3-roscdistro python3-rospkg
0 upgraded, 4 newly installed, 0 to remove and 21 not upgraded.
Need to get 68.9 kB of archives.
After this operation, 437 kB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 http://kr.archive.ubuntu.com/ubuntu jammy/universe amd64 python3-rosdep2 a
ll 0.21.0-2 [58.7 kB]
Get:2 http://packages.ros.org/ros2/ubuntu jammy/main amd64 python3-catkin-pkg al
l 1.1.0-101 [3,932 B]
Get:3 http://packages.ros.org/ros2/ubuntu jammy/main amd64 python3-roscdistro all
 1.0.1-100 [3,724 B]
Get:4 http://packages.ros.org/ros2/ubuntu jammy/main amd64 python3-rospkg all 1.
6.1-100 [2,520 B]
Fetched 68.9 kB in 1s (49.6 kB/s)
Selecting previously unselected package python3-catkin-pkg.
(Reading database ... 302237 files and directories currently installed.)
Preparing to unpack .../python3-catkin-pkg_1.1.0-101_all.deb ...
Unpacking python3-catkin-pkg (1.1.0-101) ...
Selecting previously unselected package python3-roscdistro.
Preparing to unpack .../python3-roscdistro_1.0.1-100_all.deb ...
Unpacking python3-roscdistro (1.0.1-100) ...
Selecting previously unselected package python3-rospkg.
Preparing to unpack .../python3-rospkg_1.6.1-100_all.deb ...
Unpacking python3-rospkg (1.6.1-100) ...
Selecting previously unselected package python3-rosdep2.
```

의존성 패키지를 먼저 받도록하자

의존성을 확인하고 설치한다

```
rosdep install --from-paths src --ignore-src -r -y
```

```
hanmin@Hanmin: ~/ros2_ws

ERROR: your rosdep installation has not been initialized yet. Please run:

    rosdep update

hanmin@Hanmin:~/ros2_ws$ rosdep update
reading in sources list data from /etc/ros/rosdep/sources.list.d
Hit file:///usr/share/python3-rosdep2/debian.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/osx-homebrew.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/base.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/python.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/ruby.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/releases/fuerte.yaml
Query rosdistro index https://raw.githubusercontent.com/ros/rosdistro/master/index-v4.yaml
Skip end-of-life distro "ardent"
Skip end-of-life distro "bouncy"
Skip end-of-life distro "crystal"
Skip end-of-life distro "dashing"
Skip end-of-life distro "eloquent"
Skip end-of-life distro "foxy"
Skip end-of-life distro "galactic"
Skip end-of-life distro "groovy"
Add distro "humble"
Skip end-of-life distro "hydro"
Skip end-of-life distro "indigo"
Skip end-of-life distro "iron"
Skip end-of-life distro "jade"
Add distro "jazzy"
Add distro "kilted"
Skip end-of-life distro "kinetic"
Skip end-of-life distro "lunar"
Add distro "lyrical"
Skip end-of-life distro "melodic"
Skip end-of-life distro "noetic"
Add distro "rolling"
updated cache in /home/hanmin/.ros/rosdep/sources.cache
hanmin@Hanmin:~/ros2_ws$ rosdep install --from-paths src --ignore-src -r -y
#All required rosdeps installed successfully
hanmin@Hanmin:~/ros2_ws$
```

이렇게 나오면 성공

패키지를 빌드한다.

```
colcon build --symlink-install
```

```
hanmin@Hanmin: ~/ros2_ws
hanmin@Hanmin:~/ros2_ws$ rosdep update
reading in sources list data from /etc/ros/rosdep/sources.list.d
Hit file:///usr/share/python3-rosdep2/debian.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/osx-homebrew.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/base.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/python.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/ruby.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/releases/fuerte.yaml
Query rosdistro index https://raw.githubusercontent.com/ros/rosdistro/master/index-v4.yaml
Skip end-of-life distro "ardent"
Skip end-of-life distro "bouncy"
Skip end-of-life distro "crystal"
Skip end-of-life distro "dashing"
Skip end-of-life distro "eloquent"
Skip end-of-life distro "foxy"
Skip end-of-life distro "galactic"
Skip end-of-life distro "groovy"
Add distro "humble"
Skip end-of-life distro "hydro"
Skip end-of-life distro "indigo"
Skip end-of-life distro "iron"
Skip end-of-life distro "jade"
Add distro "jazzy"
Add distro "kilted"
Skip end-of-life distro "kinetic"
Skip end-of-life distro "lunar"
Add distro "lyrical"
Skip end-of-life distro "melodic"
Skip end-of-life distro "noetic"
Add distro "rolling"
updated cache in /home/hanmin/.ros/rosdep/sources.cache
hanmin@Hanmin:~/ros2_ws$ rosdep install --from-paths src --ignore-src -r -y
#All required rosdeps installed successfully
hanmin@Hanmin:~/ros2_ws$ colcon build --symlink-install
Starting >>> my_robot_controller
Finished <<< my_robot_controller [1.73s]

Summary: 1 package finished [2.18s]
hanmin@Hanmin:~/ros2_ws$
```

특정 패키지만 빌드하려면 다음 명령을 사용한다.

```
colcon build --symlink-install --packages-select my_robot_controller
```

빌드가 완료되면 워크스페이스 환경을 현재 터미널에 적용한다.

```
source install/setup.bash
```

```
hanmin@Hanmin: ~/ros2_ws
reading in sources list data from /etc/ros/rosdep/sources.list.d
Hit file:///usr/share/python3-rosdep2/debian.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/osx-homebrew.y
aml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/base.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/python.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/ruby.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/releases/fuerte.yaml
Query rosdistro index https://raw.githubusercontent.com/ros/rosdistro/master/ind
ex-v4.yaml
Skip end-of-life distro "ardent"
Skip end-of-life distro "bouncy"
Skip end-of-life distro "crystal"
Skip end-of-life distro "dashing"
Skip end-of-life distro "eloquent"
Skip end-of-life distro "foxy"
Skip end-of-life distro "galactic"
Skip end-of-life distro "groovy"
Add distro "humble"
Skip end-of-life distro "hydro"
Skip end-of-life distro "indigo"
Skip end-of-life distro "iron"
Skip end-of-life distro "jade"
Add distro "jazzy"
Add distro "kilted"
Skip end-of-life distro "kinetic"
Skip end-of-life distro "lunar"
Add distro "lyrical"
Skip end-of-life distro "melodic"
Skip end-of-life distro "noetic"
Add distro "rolling"
updated cache in /home/hanmin/.ros/rosdep/sources.cache
hanmin@Hanmin:~/ros2_ws$ rosdep install --from-paths src --ignore-src -r -y
#All required rosdeps installed successfully
hanmin@Hanmin:~/ros2_ws$ colcon build --symlink-install
Starting >>> my_robot_controller
Finished <<< my_robot_controller [1.73s]

Summary: 1 package finished [2.18s]
hanmin@Hanmin:~/ros2_ws$ source install/setup.bash
hanmin@Hanmin:~/ros2_ws$ S
```

ROS 2 기본 환경과 현재 워크스페이스 환경을 자동 적용하려면 `~/.bashrc` 에 다음 내용을 추가할 수 있다.

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
```

```
hanmin@Hanmin: ~/ros2_ws
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/python.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/ruby.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/releases/fuerte.yaml
Query rosdistro index https://raw.githubusercontent.com/ros/rosdistro/master/index-v4.yaml
Skip end-of-life distro "ardent"
Skip end-of-life distro "bouncy"
Skip end-of-life distro "crystal"
Skip end-of-life distro "dashing"
Skip end-of-life distro "eloquent"
Skip end-of-life distro "foxy"
Skip end-of-life distro "galactic"
Skip end-of-life distro "groovy"
Add distro "humble"
Skip end-of-life distro "hydro"
Skip end-of-life distro "indigo"
Skip end-of-life distro "iron"
Skip end-of-life distro "jade"
Add distro "jazzy"
Add distro "kilted"
Skip end-of-life distro "kinetic"
Skip end-of-life distro "lunar"
Add distro "lyrical"
Skip end-of-life distro "melodic"
Skip end-of-life distro "noetic"
Add distro "rolling"
updated cache in /home/hanmin/.ros/rosdep/sources.cache
hanmin@Hanmin:~/ros2_ws$ rosdep install --from-paths src --ignore-src -r -y
#All required rosdeps installed successfully
hanmin@Hanmin:~/ros2_ws$ colcon build --symlink-install
Starting >>> my_robot_controller
Finished <<< my_robot_controller [1.73s]

Summary: 1 package finished [2.18s]
hanmin@Hanmin:~/ros2_ws$ source install/setup.bash
hanmin@Hanmin:~/ros2_ws$ Ssource /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
Ssource: command not found
hanmin@Hanmin:~/ros2_ws$ source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
hanmin@Hanmin:~/ros2_ws$
```

단, 아직 한 번도 빌드하지 않아 `install/setup.bash` 가 없는 상태라면 두 번째 줄에서 오류가 발생할 수 있다.

노드실행

예시노드가 `setup.py` 의 `console_scripts` 에 등록되 있다면 다음 명령으로 실행된다

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 run my_robot_controller controller_node
```

예상 출력은 다음과 같다.

```
[INFO] [시간] [robot_controller]: my_robot_controller 노드가 실행되었습니다.
```

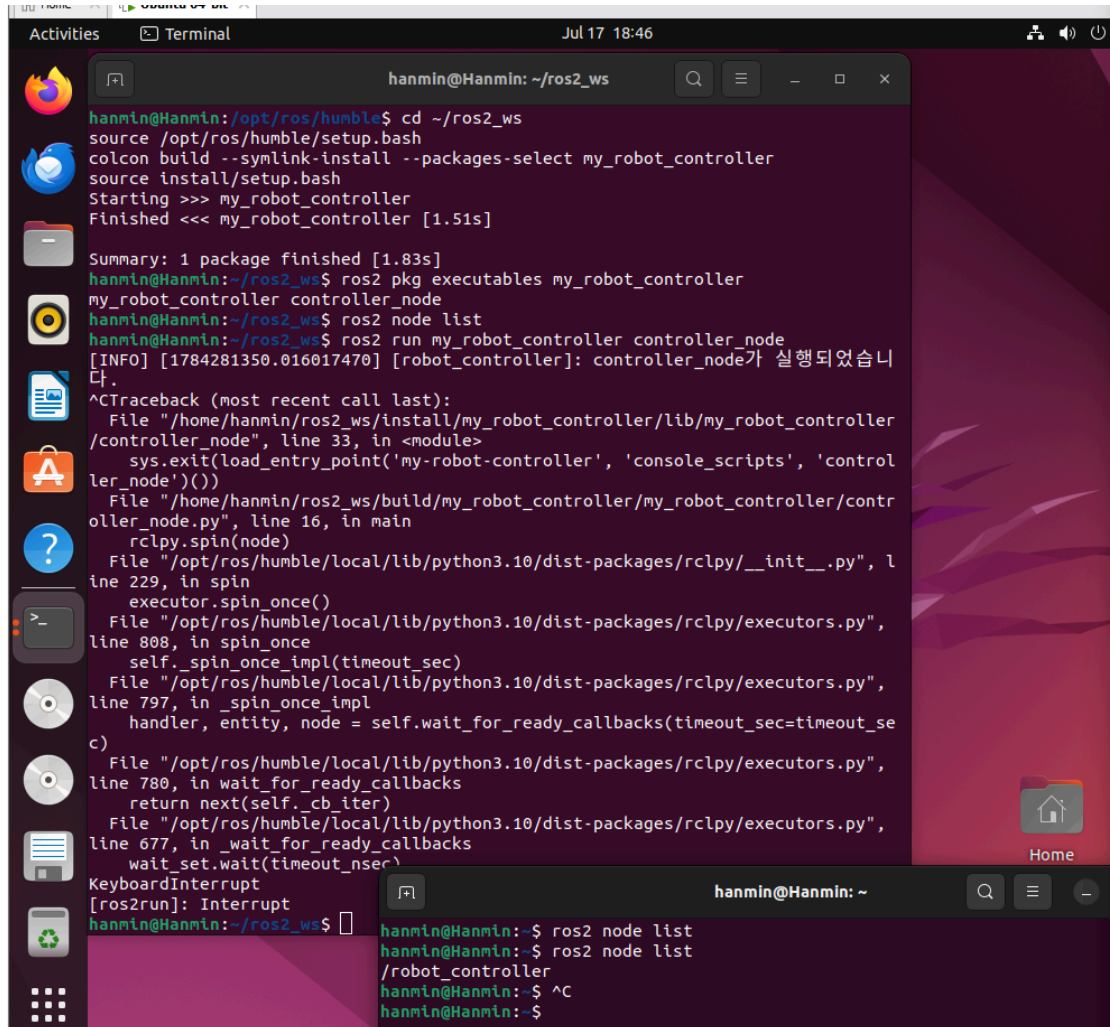
다른 터미널에서 실행 중인 노드를 확인한다.

```
ros2 node list
```

예상 출력:

```
/robot_controller
```

노드를 종료할 때는 실행 터미널에서 **Ctrl+C** 를 누른다.



```
hanmin@Hanmin:~/ros2_ws$ cd ~/ros2_ws
hanmin@Hanmin:~/ros2_ws$ source /opt/ros/humble/setup.bash
hanmin@Hanmin:~/ros2_ws$ colcon build --symlink-install --packages-select my_robot_controller
hanmin@Hanmin:~/ros2_ws$ source install/setup.bash
hanmin@Hanmin:~/ros2_ws$ Starting >>> my_robot_controller
hanmin@Hanmin:~/ros2_ws$ Finished <<< my_robot_controller [1.51s]

Summary: 1 package finished [1.83s]
hanmin@Hanmin:~/ros2_ws$ ros2 pkg executables my_robot_controller
my_robot_controller controller_node
hanmin@Hanmin:~/ros2_ws$ ros2 node list
hanmin@Hanmin:~/ros2_ws$ ros2 run my_robot_controller controller_node
[INFO] [1784281350.016017470] [robot_controller]: controller_node가 실행되었습니다.
^CTraceback (most recent call last):
  File "/home/hanmin/ros2_ws/install/my_robot_controller/lib/my_robot_controller/controller_node", line 33, in <module>
    sys.exit(load_entry_point('my-robot-controller', 'console_scripts', 'controller_node')())
  File "/home/hanmin/ros2_ws/build/my_robot_controller/my_robot_controller/controller_node.py", line 16, in main
    rclpy.spin(node)
  File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/__init__.py", line 229, in spin
    executor.spin_once()
  File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/executors.py", line 808, in spin_once
    self._spin_once_impl(timeout_sec)
  File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/executors.py", line 797, in _spin_once_impl
    handler, entity, node = self.wait_for_ready_callbacks(timeout_sec=timeout_sec)
  File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/executors.py", line 780, in wait_for_ready_callbacks
    return next(self._cb_iter)
  File "/opt/ros/humble/local/lib/python3.10/dist-packages/rclpy/executors.py", line 677, in _wait_for_ready_callbacks
    wait_set.wait(timeout_nsec)
KeyboardInterrupt
[ros2run]: Interrupt
hanmin@Hanmin:~/ros2_ws$ ros2 node list
hanmin@Hanmin:~/ros2_ws$ ros2 node list
/robot_controller
hanmin@Hanmin:~/ros2_ws$ ^C
hanmin@Hanmin:~/ros2_ws$
```

tree 프로그램 설치 및 실행

설치

```
sudo apt update
sudo apt install tree
```



```

[ros2run]: Interrupt
hanmin@Hanmin:~/ros2_ws$ sudo apt update
Hit:1 https://dl.google.com/linux/chrome-stable/deb stable InRelease
Hit:2 http://kr.archive.ubuntu.com/ubuntu jammy InRelease
Get:3 http://kr.archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Hit:4 http://kr.archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:5 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:6 http://packages.ros.org/ros2/ubuntu jammy InRelease
Fetched 128 kB in 2s (80.2 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
21 packages can be upgraded. Run 'apt list --upgradable' to see them.
hanmin@Hanmin:~/ros2_ws$ sudo apt install tree
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
tree is already the newest version (2.0.2-1).
The following packages were automatically installed and are no longer required:
  libfwupd2 libfwupdplugin5 libgcab-1.0-0 libsmbios-c2
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 21 not upgraded.
hanmin@Hanmin:~/ros2_ws$

```

설치확인:

```

cd ~/ros2_ws
tree src

```

```

hanmin@Hanmin:~/ros2_ws$ cd ~/ros2_ws
tree src
src
├── my_robot_controller
│   ├── LICENSE
│   ├── my_robot_controller
│   │   ├── controller_node.py
│   │   ├── __init__.py
│   │   └── __pycache__
│   │       ├── controller_node.cpython-310.pyc
│   │       └── __init__.cpython-310.pyc
│   ├── package.xml
│   ├── resource
│   │   └── my_robot_controller
│   ├── setup.cfg
│   ├── setup.py
│   └── test
│       ├── test_copyright.py
│       ├── test_flake8.py
│       └── test_pep257.py
5 directories, 12 files

```

빌드 후 워크스페이스 확인

```

cd ~/ros2_ws
tree -L 2

```

```
hanmin@Hanmin:~/ros2_ws$ cd ~/ros2_ws
tree -L 2
.
├── build
│   ├── COLCON_IGNORE
│   └── my_robot_controller
├── install
│   ├── COLCON_IGNORE
│   ├── local_setup.bash
│   ├── local_setup.ps1
│   ├── local_setup.sh
│   ├── _local_setup_util_ps1.py
│   ├── _local_setup_util_sh.py
│   ├── local_setup.zsh
│   ├── my_robot_controller
│   ├── setup.bash
│   ├── setup.ps1
│   ├── setup.sh
│   └── setup.zsh
├── log
│   ├── build_2026-07-17_18-25-21
│   ├── build_2026-07-17_18-30-54
│   ├── build_2026-07-17_18-37-37
│   ├── COLCON_IGNORE
│   ├── latest -> latest_build
│   └── latest_build -> build_2026-07-17_18-37-37
└── src
    └── my_robot_controller

12 directories, 13 files
hanmin@Hanmin:~/ros2_ws$
```



tree src

- src 내부를 자세히 확인
- 패키지 파일 구조 확인용

tree -L 2

- 현재 폴더에서 2단계까지만 확인
- 워크스페이스 전체 구조 확인용

빌드 디렉토리 역할

디렉토리	역할
src	사용자가 작성한 ROS2 패키지 원본 코드
build	패키지별 빌드 과정에서 생성되는 중간파일
install	빌드가 완료된 실행가능한 형태로 설치된 결과
log	빌드 및 테스트 과정의 로그

테스트 수행

패키지 테스트는 워크스페이스 루트에서 실행한다.

```
cd ~/ros2_ws
colcon test --packages-select my_robot_controller
```

테스트 결과를 자세히 확인한다.


```
colcon test-result --verbose
```

테스트 관련 오류가 발생하면 먼저 다음 명령으로 필요한 패키지가 설치되었는지 확인한다.

```
rosdep install --from-paths src --ignore-src -r -y
```

```
hanmin@Hanmin:~/ros2_ws$ cd ~/ros2_ws
colcon test --packages-select my_robot_controller
Starting >>> my_robot_controller
--- stderr: my_robot_controller

===== warnings summary =====
test/test_flake8.py::test_flake8
test/test_flake8.py::test_flake8
  Warning: SelectableGroups dict interface is deprecated. Use select.

-- Docs: https://docs.pytest.org/en/stable/warnings.html
---
Finished <<< my_robot_controller [1.45s]

Summary: 1 package finished [1.88s]
1 package had stderr output: my_robot_controller
hanmin@Hanmin:~/ros2_ws$ colcon test-result --verbose
Summary: 3 tests, 0 errors, 0 failures, 1 skipped
hanmin@Hanmin:~/ros2_ws$ rosdep install --from-paths src --ignore-src -r -y
#All required rosdeps installed successfully
hanmin@Hanmin:~/ros2_ws$
```

18. src 디렉토리 압축

과제에서는 워크스페이스 전체가 아니라 워크스페이스의 `src` 디렉토리를 압축하여 제출하도록 요구하고 있다.

18.1 zip 설치

```
sudo apt update
sudo apt install zip
```

18.2 src 압축

워크스페이스 루트에서 실행한다.

```
cd ~/ros2_ws
zip -r ros2_ws_src.zip src
```

```

hanmin@Hanmin:~/ros2_ws$ cd ~/ros2_ws
zip -r ros2_ws_src.zip src
  adding: src/ (stored 0%)
  adding: src/my_robot_controller/ (stored 0%)
  adding: src/my_robot_controller/resource/ (stored 0%)
  adding: src/my_robot_controller/resource/my_robot_controller (stored 0%)
  adding: src/my_robot_controller/test/ (stored 0%)
  adding: src/my_robot_controller/test/test_flake8.py (deflated 40%)
  adding: src/my_robot_controller/test/test_copyright.py (deflated 41%)
  adding: src/my_robot_controller/test/test_pep257.py (deflated 38%)
  adding: src/my_robot_controller/package.xml (deflated 50%)
  adding: src/my_robot_controller/my_robot_controller/ (stored 0%)
  adding: src/my_robot_controller/my_robot_controller/__pycache__/ (stored 0%)
  adding: src/my_robot_controller/my_robot_controller/__pycache__/controller_node.cpython-310.pyc (deflated 39%)
  adding: src/my_robot_controller/my_robot_controller/__pycache__/__init__.cpython-310.pyc (deflated 30%)
  adding: src/my_robot_controller/my_robot_controller/controller_node.py (deflated 40%)
  adding: src/my_robot_controller/my_robot_controller/__init__.py (stored 0%)
  adding: src/my_robot_controller/setup.py (deflated 57%)
  adding: src/my_robot_controller/setup.cfg (deflated 34%)
  adding: src/my_robot_controller/LICENSE (deflated 65%)
hanmin@Hanmin:~/ros2_ws$

```

압축 파일 확인:

```
ls -lh ros2_ws_src.zip
```

```
hanmin@Hanmin: ~/ros2_ws
Hit:5 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:6 http://packages.ros.org/ros2/ubuntu jammy InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
21 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
zip is already the newest version (3.0-12build2).
zip set to manually installed.
The following packages were automatically installed and are no longer required:
  libfwupd2 libfwupdplugin5 libgcb-1.0-0 libsbios-c2
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 21 not upgraded.
hanmin@Hanmin:~/ros2_ws$ cd ~/ros2_ws
zip -r ros2_ws_src.zip src
  adding: src/ (stored 0%)
  adding: src/my_robot_controller/ (stored 0%)
  adding: src/my_robot_controller/resource/ (stored 0%)
  adding: src/my_robot_controller/resource/my_robot_controller (stored 0%)
  adding: src/my_robot_controller/test/ (stored 0%)
  adding: src/my_robot_controller/test/test_flake8.py (deflated 40%)
  adding: src/my_robot_controller/test/test_copyright.py (deflated 41%)
  adding: src/my_robot_controller/test/test_pep257.py (deflated 38%)
  adding: src/my_robot_controller/package.xml (deflated 50%)
  adding: src/my_robot_controller/my_robot_controller/ (stored 0%)
  adding: src/my_robot_controller/my_robot_controller/__pycache__/ (stored 0%)
  adding: src/my_robot_controller/my_robot_controller/__pycache__/controller_node.cpython-310.pyc (deflated 39%)
  adding: src/my_robot_controller/my_robot_controller/__pycache__/__init__.cpython-310.pyc (deflated 30%)
  adding: src/my_robot_controller/my_robot_controller/controller_node.py (deflated 40%)
  adding: src/my_robot_controller/my_robot_controller/__init__.py (stored 0%)
  adding: src/my_robot_controller/setup.py (deflated 57%)
  adding: src/my_robot_controller/setup.cfg (deflated 34%)
  adding: src/my_robot_controller/LICENSE (deflated 65%)
hanmin@Hanmin:~/ros2_ws$ ls -lh ros2_ws_src.zip
-rw-rw-r-- 1 hanmin hanmin 11K Jul 17 18:56 ros2_ws_src.zip
hanmin@Hanmin:~/ros2_ws$
```

압축 내용 확인:

```
unzip -l ros2_ws_src.zip
```

```
unzip -l ros2_ws_src.zip
```

이 명령어는 `ros2_ws_src.zip` 이라는 압축 파일의 압축을 풀지 않고, 그 안에 어떤 파일과 폴더들이 들어있는지 목록만 미리 확인할 때 사용하는 명령어입니다.

1. 명령어 뜯어보기

- `unzip` : `.zip` 형식의 압축 파일을 다루는 리눅스 명령어입니다.
- `-l` : "List"의 약자로, 압축을 실제로 풀지는(extract) 않고 내부 파일 목록만 화면에 출력해달라는 옵션입니다.
- `ros2_ws_src.zip` : 대상이 되는 압축 파일 이름입니다. (이름으로 보아 ROS 2 작업 공간의 소스코드 폴더인 `src` 를 압축해 둔 파일 같네요!)

2. 실행하면 화면에 어떻게 나오나요?

명령어를 실행하면 터미널에 아래처럼 압축 파일 내의 파일 크기, 생성 날짜/시간, 그리고 폴더 및 파일 경로가 표 형태로 깔끔하게 출력됩니다.

```
hanmin@Hanmin:~/ros2_ws$ unzip -l ros2_ws_src.zip
Archive:  ros2_ws_src.zip
  Length      Date    Time    Name
-----
0         2026-07-17  11:26    src/
0         2026-07-17  18:37    src/my_robot_controller/
0         2026-07-17  11:26    src/my_robot_controller/resource/
0         2026-07-17  11:26    src/my_robot_controller/resource/my_robot_controll
er
0         2026-07-17  11:26    src/my_robot_controller/test/
884       2026-07-17  11:26    src/my_robot_controller/test/test_flake8.py
962       2026-07-17  11:26    src/my_robot_controller/test/test_copyright.py
803       2026-07-17  11:26    src/my_robot_controller/test/test_pep257.py
648       2026-07-17  11:26    src/my_robot_controller/package.xml
0         2026-07-17  18:42    src/my_robot_controller/my_robot_controller/
0         2026-07-17  18:42    src/my_robot_controller/my_robot_controller/__pyca
che__/
928       2026-07-17  18:42    src/my_robot_controller/my_robot_controller/__pyca
che__/controller_node.cpython-310.pyc
171       2026-07-17  18:42    src/my_robot_controller/my_robot_controller/__pyca
che__/__init__.cpython-310.pyc
404       2026-07-17  18:34    src/my_robot_controller/my_robot_controller/contro
ller_node.py
0         2026-07-17  11:26    src/my_robot_controller/my_robot_controller/__init
___.py
835       2026-07-17  18:37    src/my_robot_controller/setup.py
107       2026-07-17  11:26    src/my_robot_controller/setup.cfg
11358     2026-07-17  11:26    src/my_robot_controller/LICENSE
-----
17100
18 files
```

`build`, `install`, `log` 디렉토리는 빌드할 때 다시 생성할 수 있고 용량이 크므로, 과제 지시가 `src` 압축인 경우 포함하지 않는다.

프로젝트 제출 디렉토리 구성

프로젝트 루트에 2/4 디렉토리를 생성한다.

```
cd ~/프로젝트_루트
mkdir -p 2/4
```

조사 문서를 복사한다.

```
cp ~/ros2_ws/4_ros2_package.md 2/4/
```

압축 파일을 복사한다.

```
cp ~/ros2_ws/ros2_ws_src.zip 2/4/
```

최종 제출 구조는 다음과 같이 구성한다.

```
프로젝트_루트/
├── 2/
│   └── 4/
│       ├── 4_ros2_package.md
│       └── ros2_ws_src.zip
```

실습 화면 캡처가 필요한 경우 다음 항목을 추가하면 평가자가 수행 과정을 확인하기 쉽다.

- `printenv ROS_DISTRO` 결과
- `ros2 pkg create` 실행 결과
- `tree src` 실행 결과
- `colcon build --symlink-install` 성공 결과
- `ros2 run my_robot_controller controller_node` 실행 결과
- `ros2 node list` 결과

20. 전체 실습 명령 요약

아래 명령을 순서대로 실행하면 워크스페이스 생성부터 패키지 빌드까지 진행할 수 있다.

```
# 1. ROS 2 Humble 환경 적용
source /opt/ros/humble/setup.bash

# 2. 필요한 도구 설치
sudo apt update
sudo apt install -y python3-colcon-common-extensions python3-rosdep tree zip

# 3. 워크스페이스 생성
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws
```

```

# 4. 빈 워크스페이스 빌드 확인
colcon build

# 5. Python 패키지 생성
cd ~/ros2_ws/src
ros2 pkg create \
  --build-type ament_python \
  --dependencies rclpy \
  --license Apache-2.0 \
  my_robot_controller

# 6. 패키지 구조 확인
cd ~/ros2_ws
tree src

# 7. 의존성 설치
rosdep install --from-paths src --ignore-src -r -y

# 8. 패키지 빌드
colcon build --symlink-install

# 9. 워크스페이스 환경 적용
source install/setup.bash

# 10. 패키지 확인
ros2 pkg list | grep my_robot_controller

# 11. 노드 실행
ros2 run my_robot_controller controller_node

# 12. src 압축
cd ~/ros2_ws
zip -r ros2_ws_src.zip src

```

패키지 생성 직후에는 `controller_node.py` 와 `setup.py` 의 `console_scripts` 등록이 자동으로 포함되지 않을 수 있다. 노드 실행 실습까지 진행하려면 본 문서의 예시처럼 두 항목을 직접 추가해야 한다.

21. 자주 발생하는 오류와 해결 방법

21.1 ros2 명령어가 없는 경우

오류:

```
ros2: command not found
```

해결:

```
source /opt/ros/humble/setup.bash
```

자동 적용:

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc  
source ~/.bashrc
```

21.2 colcon 명령어가 없는 경우

오류:

```
colcon: command not found
```

해결:

```
sudo apt update  
sudo apt install python3-colcon-common-extensions
```

21.3 패키지를 찾지 못하는 경우

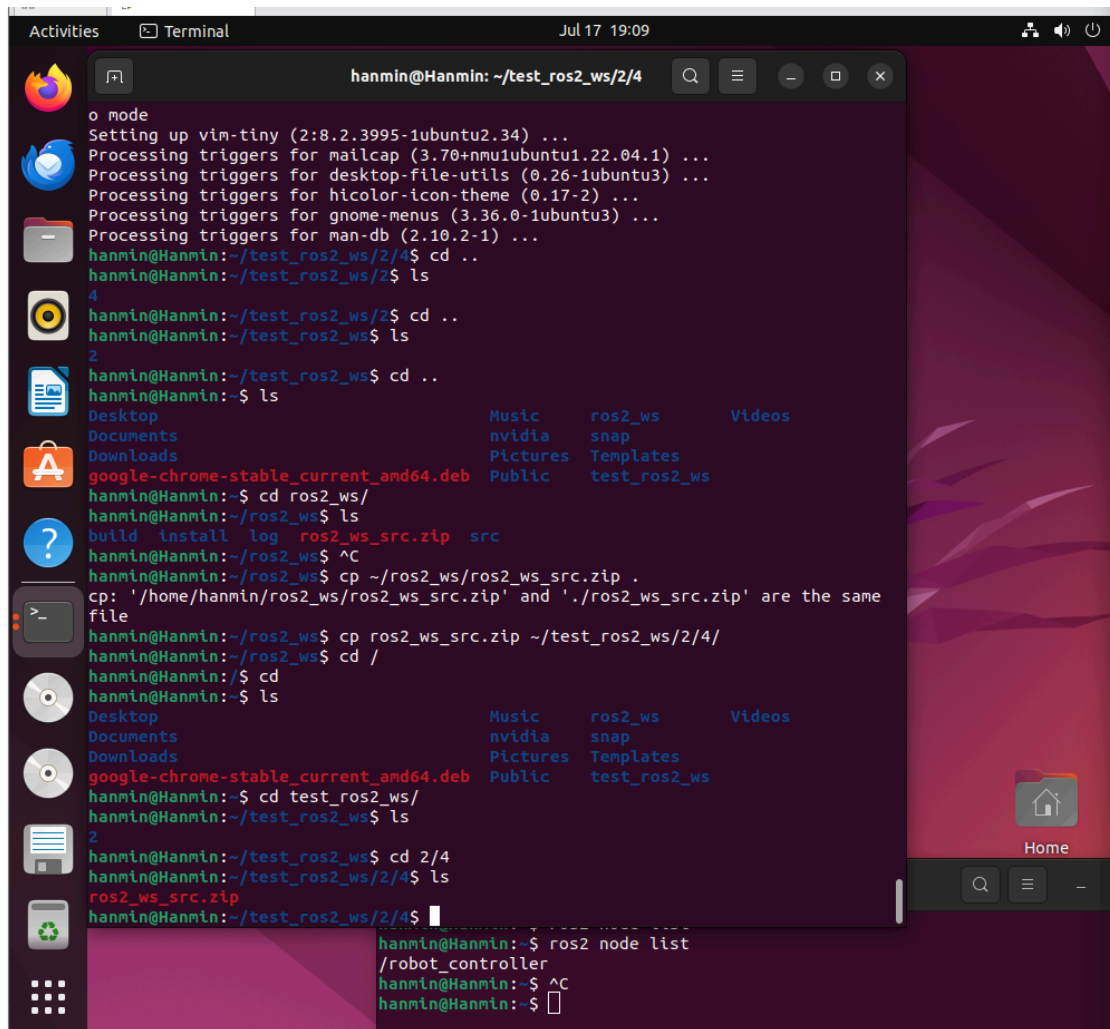
오류:

```
Package 'my_robot_controller' not found
```

해결:

```
cd ~/ros2_ws  
colcon build --symlink-install  
source install/setup.bash
```

새 터미널을 열었다면 다시 source해야 한다.



21.4 실행 파일을 찾지 못하는 경우

오류:

```
No executable found
```

확인할 사항:

1. `setup.py`의 `entry_points`에 실행 항목이 등록되어 있는지 확인한다.
2. Python 파일에 `main()` 함수가 존재하는지 확인한다.
3. 패키지를 다시 빌드한다.
4. `install/setup.bash`를 다시 source한다.

```

cd ~/ros2_ws
colcon build --symlink-install --packages-select my_robot_controller
source install/setup.bash

```

21.5 build 결과가 이전 상태로 남은 경우


```
cd ~/ros2_ws
rm -rf build install log
colcon build --symlink-install
source install/setup.bash
```

이 명령은 빌드 결과를 삭제하므로 반드시 워크스페이스 루트에서 정확히 실행해야 한다. `src` 디렉토리는 삭제하지 않는다.

22. 결론

본 실습에서는 ROS 2 Humble 환경에서 `ros2_ws` 워크스페이스와 `src` 디렉토리를 생성하고, `ros2 pkg create` 명령을 이용하여 `ament_python` 기반의 `my_robot_controller` 패키지를 생성하였다.

`colcon`은 워크스페이스 내부 패키지의 검색, 의존성 순서 결정, 빌드 및 테스트를 담당한다. `ament_python`은 Python 기반 ROS 2 패키지를 설치하기 위한 빌드 유형이며, `rclpy`는 Python 코드에서 노드, 토픽, 서비스, 액션, 파라미터 등의 ROS 2 기능을 사용할 수 있게 한다.

또한 `package.xml`은 패키지의 메타데이터, 의존성 및 빌드 유형을 기록하고, `setup.py`는 Python 모듈의 설치와 실행 명령을 정의한다. 빌드 후 생성되는 `build`, `install`, `log` 디렉토리의 용도를 확인하였으며, 제출을 위해 `src` 디렉토리를 별도의 ZIP 파일로 압축하는 방법도 실습하였다.

23. 참고 자료

1. ROS 2 Humble Documentation, Creating a workspace

<https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Creating-A-Workspace/Creating-A-Workspace.html>

2. ROS 2 Humble Documentation, Using colcon to build packages

<https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Colcon-Tutorial.html>

3. ROS 2 Humble Documentation, Creating a package

<https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Creating-Your-First-R0S2-Package.html>

4. ROS 2 Humble Documentation, Writing a simple publisher and subscriber with Python

<https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Publisher-And-Subscriber.html>

5. ROS 2 Humble Documentation, About the build system

<https://docs.ros.org/en/humble/Concepts/Advanced/About-Build-System.html>

6. ROS 2 Humble Documentation, Configuring environment

<https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Configuring-R0S2-Environment.html>

7. rclpy API Documentation

<https://docs.ros.org/en/humble/p/rclpy/>